Báo cáo COS30017 Assignment 2 – Fake Uber

Nov 8, 2026 · @bchung

1. Giới thiệu

Fake Uber là một ứng dụng Android chạy được, làm bản thử nghiệm (proof of concept) cho dịch vụ thuê xe theo tuần: app hiện năm chiếc xe, mỗi lần một xe, cho phép thuê 1–7 ngày với số dư 500 credit, và vượt qua cả 10/10 test giao diện Espresso.

Đề bài yêu cầu một app thuê xe đơn giản cho một công ty mở rộng sang cho thuê xe cũ và mới theo tuần. Thầy dặn logic chỉ cần đơn giản, trọng tâm là UI/UX, nên phần lớn thời gian thiết kế dành cho nghiên cứu, vẽ phác thảo và accessibility trước khi viết code.

Mục	Chi tiết
Tên app	Fake Uber (package `com.example.fakeuber`)
Ngôn ngữ và bộ giao diện	Kotlin, XML Views với ViewBinding
Màn hình	2 Activity: `MainActivity` (xem xe) và `RentActivity` (thuê xe)
Nền tảng	compileSdk / targetSdk 37, minSdk 24 (Android 7.0, khoảng 99,5% thiết bị)
Dữ liệu	5 xe chỉ năm trong bộ nhớ, không lưu xuống đĩa
Đơn vị tiền	Credit (1 credit = 10.000đ)
Kiểm thử	10 test Espresso, tất cả đạt trong 43 giây

App đáp ứng mọi yêu cầu của đề: hiện từng xe một với nút Next, thông tin năm dưới ảnh, thuê xe bằng Parcelable truyền qua Intent, số dư credit với hạn mức 400 credit mỗi đơn, hành vi của Save và Back, thông báo bằng Snackbar, yêu thích, tìm kiếm, sắp xếp, công tắc dark mode trên cả hai màn, ít nhất hai widget không phải TextView, style dùng lại và kiểm tra lỗi.

2. User story và use case

Hai user story bao trùm hai nhu cầu cốt lõi của app: chọn được xe phù hợp và thuê xe mà không tiêu quá số tiền cho phép; mỗi story có một use case tương ứng đi qua từng bước, kể cả các nhánh lỗi. Mọi tiêu chí chấp nhận bên dưới đều được kiểm tra bằng test Espresso hoặc test tay ở mục 8.

US1: Xem, tìm và so sánh xe

> *Là một khách hàng cần thuê xe ngắn ngày, tôi muốn thấy giá, đánh giá, năm sản xuất và số km của từng xe ngay dưới ảnh, đồng thời tìm kiếm, sắp xếp và lưu xe yêu thích, để chọn đúng xe trước khi bắt đầu đặt.*

Tiêu chí chấp nhận:

1. Màn Home hiện từng xe một với hãng, model, năm, đánh giá, số km và giá mỗi ngày dưới ảnh, kèm bộ đếm vị trí "1 / 5".
2. Next chuyển sang xe kế tiếp và quay vòng; xe đã thuê bị bỏ qua.
3. Gõ vào ô tìm kiếm chỉ giữ lại xe có hãng hoặc model khớp, không phân biệt hoa thường. Không khớp thì hiện "No cars match “xyz”" kèm nút Clear search.
4. Menu Sort có Rating (cao xuống thấp), Year (mới nhất trước) và Cost (rẻ nhất trước), đánh dấu lựa chọn đang dùng và sắp xếp trên một bản sao, không bao giờ trên dữ liệu gốc.
5. Chạm trái tim hoặc nhấn giữ ảnh để thêm xe vào Favourites; chạm lần nữa để bỏ.

US2: Thuê xe trong hạn mức credit, kèm giấy tờ

> *Là một khách hàng có 500 credit, tôi muốn thấy tổng tiền và số dư sau khi thuê trước khi xác nhận, và thêm ảnh CCCD lẫn bằng lái ngay tại đó, để không bao giờ đặt một chiếc xe tôi không đủ tiền hoặc không được phép lái.*

Tiêu chí chấp nhận:

1. Số dư credit luôn hiện ở header của cả Home và Rent.
2. Ở màn Rent, tổng tiền (giá × số ngày), số dư sau khi thuê và thanh hạn mức 400 credit cập nhật ngay khi kéo Slider 1–7 ngày.
3. Save bị từ chối, kèm lý do hiện ngay tại chỗ, nếu tổng tiền vượt 400, vượt số dư, hoặc thiếu ảnh giấy tờ. Người dùng vẫn ở lại màn Rent.
4. Save hợp lệ thì quay về Home, trừ tổng tiền, xe rời khỏi Available now, hiện trong My rentals và có Snackbar "… booked".
5. Back không lưu gì và báo "Rental not saved". Cancel booking hoàn lại credit và đưa xe trở lại danh sách.

UC1: Thuê một chiếc xe (hiện thực US2)

Mục	Mô tả
Tác nhân	Khách hàng
Mục tiêu	Thuê xe 1–7 ngày mà không vi phạm quy tắc credit
Điều kiện trước	Màn Home đang mở và còn ít nhất một xe
Thành công	Đã trừ credit, xe nằm trong My rentals và rời Available now
Huỷ	Không có gì thay đổi; số dư giữ nguyên

Luồng chính

1. Khách bấm **Next** tới khi thấy xe muốn thuê, rồi bấm **Rent**.
2. App mở màn Rent và truyền xe dưới dạng Parcelable trong một Intent tường minh.
3. App hiện ảnh, thông tin, Slider ở mức 1 ngày, thẻ Credits và hai ô giấy tờ trống.
4. Khách kéo Slider; tổng tiền, số dư sau khi thuê và thanh hạn mức cập nhật ngay.
5. Khách chạm **Driver ID Photo** và **Driver Licence Photo**, chọn ảnh cho từng ô trong Photo Picker của hệ thống. Mỗi ô hiện dấu tích.
6. Khách bấm **Save · N credits**.
7. App kiểm tra: đủ cả hai ảnh, tổng ⩽ 400, tổng ⩽ số dư.
8. App quay về Home, trừ credit, chuyển xe vào My rentals và hiện "Toyota Vios 1.5G booked · 3 days · 240 credits".

Luồng thay thế

- **7a. Tổng vượt 400:** thanh hạn mức chuyển đỏ và thẻ Credits báo "This rental costs 440 credits. One rental can't go over 400…". Khách chọn ít ngày hơn và quay lại bước 6.
- **7b. Tổng vượt số dư:** thẻ báo "Not enough credits. You have 170, this rental needs 180…". Khách chọn ít ngày hơn hoặc bấm Back.
- **7c. Thiếu ảnh:** ô trống chuyển đỏ với chữ "Required" và Snackbar "Can't save yet". Khách thêm ảnh rồi quay lại bước 6.
- **3–7. Bấm Back** (nút Back, mũi tên hoặc back của hệ thống): app về Home, không thay đổi gì, báo "Rental not saved".
- **Sau bước 8. Cancel booking:** app hoàn credit, đưa xe về danh sách và báo "Booking cancelled · N credits refunded".

UC2: Tìm, sắp xếp và lưu xe yêu thích (hiện thực US1)

Mục	Mô tả
Tác nhân	Khách hàng
Mục tiêu	Tìm xe theo tên hoặc tiêu chí và lưu lại để xem sau
Điều kiện trước	Màn Home đang mở
Thành công	Xe xuất hiện trong dải Favourites với trái tim tô đầy

Luồng chính

1. Khách gõ một phần tên hoặc model vào ô tìm kiếm, ví dụ "toyota".
2. App lọc ngay khi gõ và bộ đếm đổi thành "1 / 2".
3. Khách xoá ô tìm, mở **Sort** và chọn "Cost · lowest first".
4. App sắp xếp một bản sao của danh sách và đánh dấu lựa chọn đang dùng; danh sách gốc không đổi.
5. Khách chạm trái tim trên ảnh.
6. App tô đầy trái tim và thêm một chip (ảnh nhỏ, model, giá) vào Favourites; bộ đếm hiện "1 saved".

Luồng thay thế

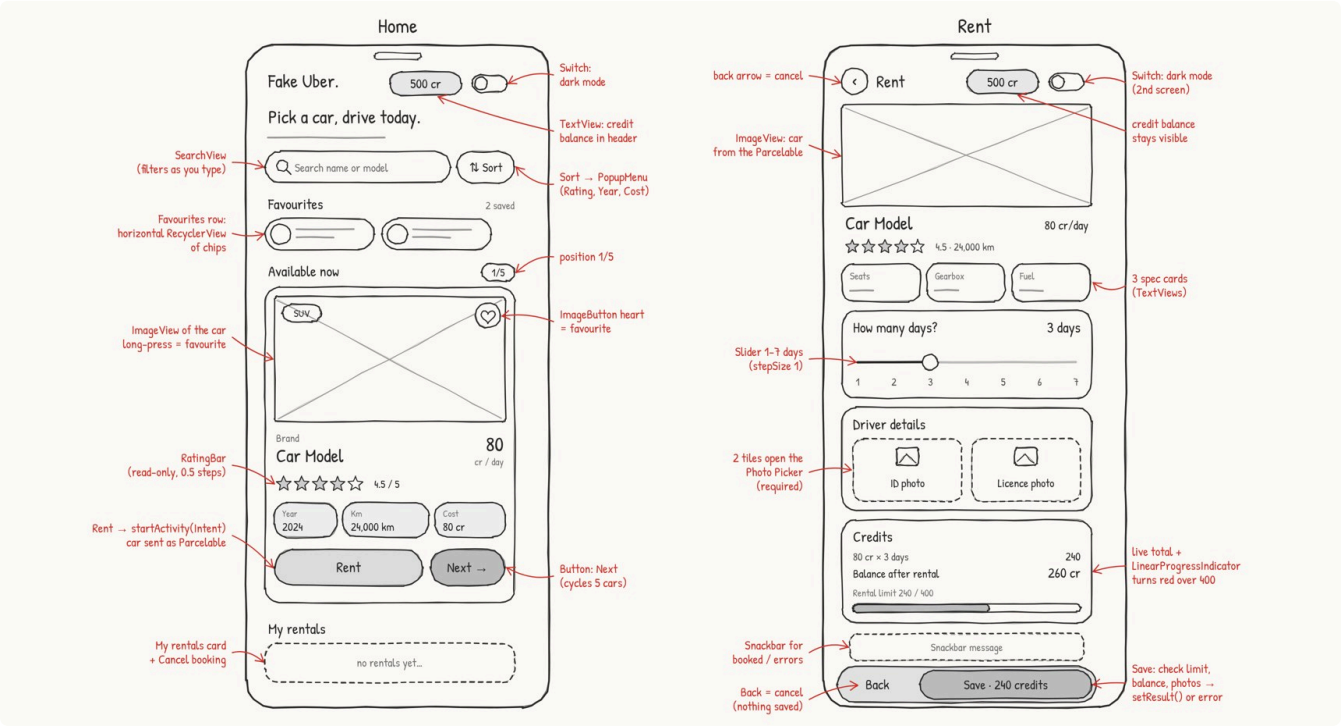
- **2a. Không có kết quả:** app hiện "No cars match “xyz”", bộ đếm "0 / 0" và nút Clear search.
- **5a. Nhấn giữ ảnh:** kết quả giống bước 6, là một lối tắt.
- **5b. Xe đã là yêu thích:** chạm trái tim sẽ bỏ xe khỏi Favourites.
- **Sau bước 6. Chạm vào chip:** thẻ nhảy tới xe đó; nếu xe đang được thuê, Snackbar giải thích vì sao không hiện được.

3. Hai bản phác thảo layout và thiết kế được chọn

Layout A được chọn vì mọi thao tác đều là nút nhìn thấy được, nằm trong tầm ngón cái, và nó ngăn lỗi credit từ trước thay vì báo lỗi sau khi bấm Save. Cả hai bản phác đều cố ý ở mức low-fi: hộp xám, ô ảnh gạch chéo và ghi chú đỏ ghi tên widget Android đứng sau mỗi thành phần.

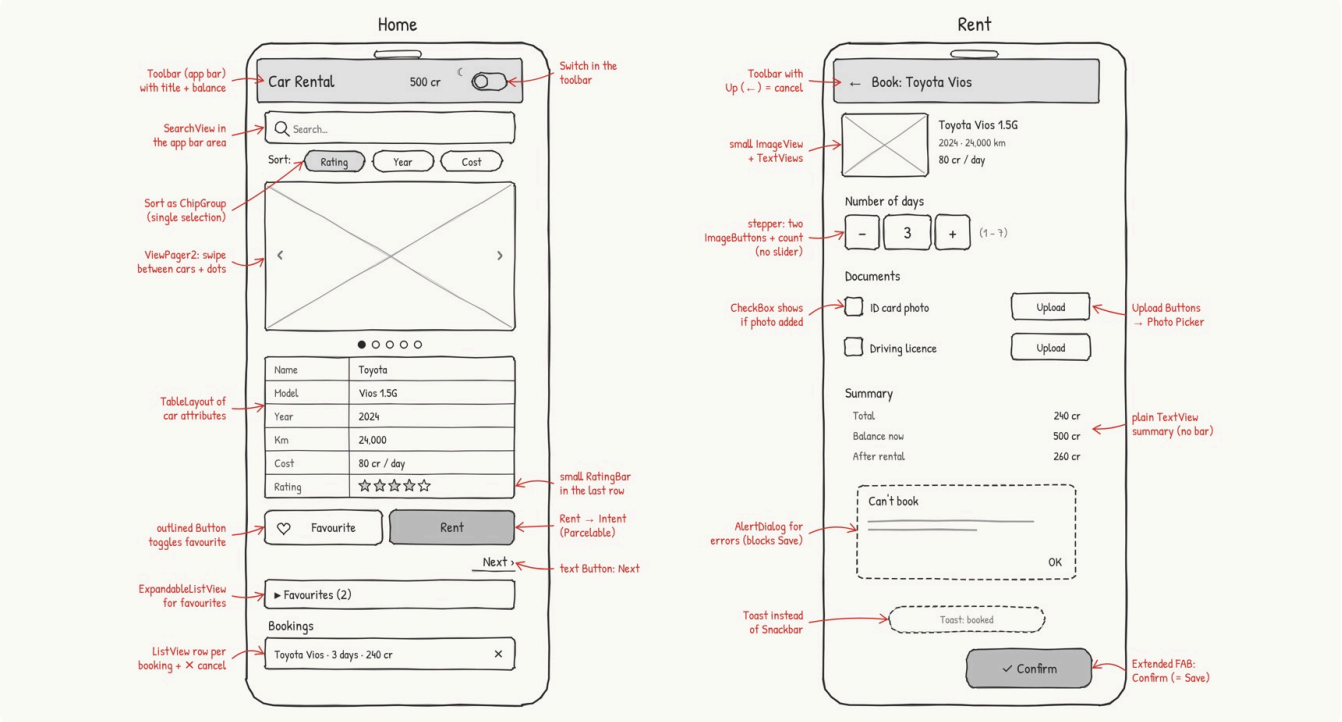
Layout A: một cột cuộn, mỗi lần một thẻ xe. Màn Home xếp các khối có tiêu đề theo một cột: header có số dư credit và Switch dark mode, ô tìm kiếm luôn hiện kèm nút Sort, dải Favourites, một thẻ xe với nút Rent và Next, rồi My rentals. Màn Rent lặp lại thông tin xe,

sau đó là ba thẻ theo thứ tự ra quyết định: số ngày (Slider), giấy tờ người lái (hai ô ảnh) và credit (tổng tiền trực tiếp và thanh hạn mức), kèm thanh Back/Save cố định ở đáy.



Hình 1. Phác thảo low-fi Layout A (Home và Rent) kèm ghi chú widget.

Layout B: thanh app bar, lướt ngang và bảng. Toolbar chứa tiêu đề, số dư và Switch; xe được lướt ngang trong ViewPager2 có chấm chỉ vị trí; sắp xếp bằng ChipGroup; thông tin năm trong TableLayout; yêu thích thu gọn trong ExpandableListView và đơn thuê trong ListView. Màn Rent dùng bộ tăng giảm −/+ thay vì Slider, nút Upload kèm CheckBox, phần tóm tắt dạng chữ, AlertDialog để báo lỗi, Toast để xác nhận và Extended FAB để chốt đơn.



Hình 2. Phác thảo low-fi Layout B (Home và Rent) kèm ghi chú widget.

Thành phần	Layout A	Layout B
Chuyển giữa các xe	Nút Next + bộ đếm "1 / 5"	Lướt ViewPager2 + chấm + nút chữ Next
Sắp xếp	Nút mở PopupMenu	ChipGroup luôn hiện
Thông tin xe	Ô chỉ số + RatingBar dưới ảnh	Các dòng TableLayout + RatingBar nhỏ
Yêu thích	Dải chip ngang (RecyclerView)	ExpandableListView
Đơn thuê	Thẻ My rentals có Cancel booking	Dòng ListView có dấu ×
Số ngày	Slider 1–7	Bộ tăng giảm: hai ImageButton + số
Giấy tờ người lái	Hai ô ảnh lớn	Nút Upload + CheckBox
Chi phí	Thẻ Credits có thanh hạn mức trực tiếp	TextView tóm tắt
Báo lỗi	Thông báo tại chỗ + Snackbar	AlertDialog
Xác nhận	Snackbar	Toast

Cả hai layout đều đáp ứng mọi yêu cầu của đề, kể cả có ít nhất hai widget không phải TextView (A: RatingBar, Slider, Switch; B: RatingBar, bộ tăng giảm, Switch). Lựa chọn cuối cùng dựa trên các tiêu chí sau:

Tiêu chí	Tốt hơn	Lý do
Dễ nhận ra cách chuyển xe	A	Nút Next và bộ đếm nhìn thấy được; thao tác lướt bị ẩn và các chấm là gợi ý yếu (Budiu, 2018)
Ngăn thuê vượt hạn mức	A	Tổng tiền và thanh hạn mức cảnh báo trước khi Save (heuristic số 5 của Nielsen, ngăn lỗi)
Chất lượng thông báo lỗi	A	Lỗi hiện tại chỗ với con số cụ thể; hộp thoại modal chặn cả màn hình (Neusesser & Sunwall, 2023)
Phản hồi làm được đúng như bản vẽ	A	Snackbar nằm ở đáy và theo theme; Toast dạng chữ không đặt được vị trí từ Android 11

Tiêu chí	Tốt hơn	Lý do
Chọn số ngày	A	Một lần kéo trên dải 1–7 nhìn thấy được, thay vì tối đa 6 lần chạm vào nút −/+ nhỏ (Parhi và cộng sự, 2006)
Test Espresso	A	Chỉ cần click và kiểm tra chữ; lướt và hộp thoại cần xử lý thêm
Số lần chạm để sắp xếp	B	1 lần chạm chip thay vì 2 (mở menu, chọn)
Mật độ thông tin	B	Bảng hiện mọi thuộc tính cùng lúc

Ưu điểm của B nhỏ với một app chỉ có năm xe và ba kiểu sắp xếp, nên Layout A trở thành thiết kế cuối cùng. App hoàn chỉnh bám sát Layout A; thay đổi có chủ ý duy nhất là thẻ thông số thứ ba ở màn Rent hiện **Type** (loại xe) thay vì **Gearbox** như bản phác, vì cả năm xe đều số tự động nên thẻ đó lúc nào cũng hiện cùng một giá trị.

4. Nghiên cứu UI/UX và các lựa chọn thiết kế

Mọi quyết định về giao diện đều xuất phát từ một kết quả nghiên cứu, và thay đổi lớn nhất mà nghiên cứu buộc phải làm là bảng màu: ba màu trong thiết kế đầu tiên không đạt chuẩn tương phản WCAG và đã được thay trước khi viết code.

Mỗi vùng màn hình trả lời một câu hỏi. Home trả lời "thuê xe nào?", Rent trả lời "thuê bao lâu, có đủ tiền không?". Các thẻ ở màn Rent xếp theo thứ tự ra quyết định (số ngày, giấy tờ, rồi credit), nên tổng tiền luôn nằm cạnh điều khiển làm thay đổi nó.

Ngăn lỗi thay vì báo lỗi. Thẻ Credits hiện tổng tiền, số dư sau khi thuê và thanh đo hạn mức 400 credit ngay khi người dùng còn đang chọn. Lỗi vượt hạn mức và không đủ credit hiện trực tiếp khi kéo Slider, kèm con số cụ thể và cách sửa ("Pick fewer days or press Back to cancel"). Thiếu ảnh chỉ báo sau khi bấm Save, để không cảnh báo người dùng khi họ chưa kịp thêm ảnh.

Phản hồi đáng tin cậy. Mọi kết quả (đã đặt, không lưu, đã huỷ và hoàn tiền, chưa lưu được) đều hiện Snackbar hai dòng: tiêu đề in đậm và một dòng chi tiết. Chọn Snackbar thay vì Toast vì Snackbar theo màu theme, được TalkBack đọc và có thể nằm ngay trên thanh Back/Save (Material Components for Android, n.d.); còn `setGravity()` của Toast dạng chữ không còn tác dụng từ Android 11 (Android Developers, n.d.-f).

Giấy tờ mà không cần xin quyền. Hai ô giấy tờ mở Photo Picker của hệ thống (`PickVisualMedia`), không cần quyền truy cập bộ nhớ: người dùng chỉ chia sẻ đúng tấm ảnh họ chọn (Android Developers, n.d.-e). Dấu tích và ảnh thu nhỏ xác nhận từng ảnh đã thêm.

Màu sắc dễ đọc. Các màu được kiểm tra theo mức tương phản tối thiểu của WCAG 2.2: 4,5:1 cho chữ thường và 3:1 cho chữ lớn và thành phần giao diện (W3C, 2023).

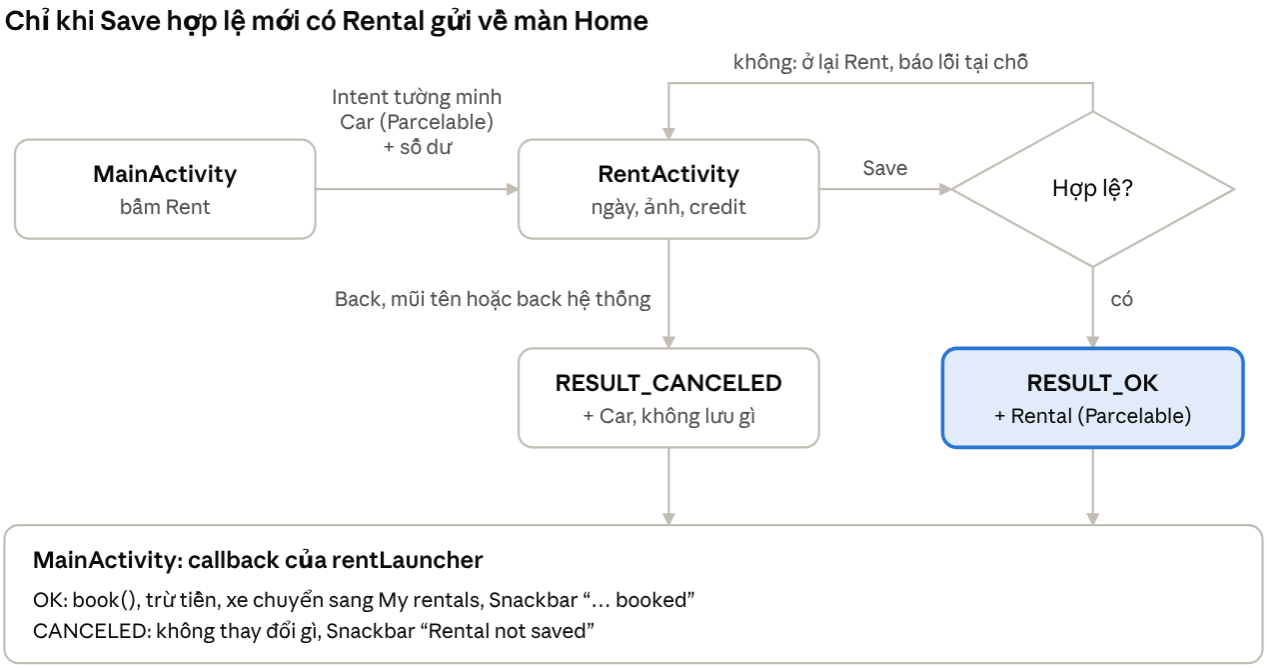
Vai trò màu	Thiết kế đầu	Cuối cùng	Lý do
Cam thương hiệu (nút, Slider, điểm nhấn)	#F97316 với chữ trắng	#EA580C với chữ **tối**	Chữ trắng trên cam chỉ khoảng 2,8:1; chữ tối trên #EA580C đạt 4,9:1
Cam dùng cho chữ nhỏ ("Clear search")	#F97316	#C2410C	5,2:1 trên nền thẻ
Chữ phụ màu xám	#716C65	#625D57	5,8:1 trên nền trang (trước đó dưới 4,5:1)
Xanh "Rented"	#2F9E5E	#1F7A45 (sáng), #4ADE80 (tối)	5,2:1; giao diện tối cần màu xanh sáng hơn
Đỏ báo lỗi	#E11D2E	#B91C1C (sáng), #F87171 (tối)	5,3:1 trong hộp lỗi hồng; 6:1 trên nền tối

Vùng chạm tối thiểu 48dp. Trái tim, công tắc, mũi tên back, ô giấy tờ và mọi nút đều tối thiểu 48 × 48dp theo chuẩn Android, các nút chính cao 56dp để thao tác một tay (Parhi và cộng sự, 2006).

Hỗ trợ trình đọc màn hình. Ảnh xe, trái tim, ô giấy tờ và nút Cancel booking có content description nêu tên xe hoặc hành động. Tiêu đề mục được đánh dấu là heading, hộp lỗi là live region để TalkBack đọc khi nội dung đổi, và thao tác nhấn giữ ảnh có nhãn TalkBack riêng ("add or remove favourite").

5. Hiện thực: kiến trúc, Intent và Parcelable

App gồm hai Activity chỉ giao tiếp qua Intent: `MainActivity` gửi xe được chọn sang `RentActivity` dưới dạng Parcelable, và `RentActivity` trả về hoặc một `Rental` (Parcelable, khi Save) hoặc một lệnh huỷ (khi Back). Toàn bộ trạng thái nằm trong hai ViewModel, nên vẫn còn sau khi bật/tắt dark mode mà vẫn chỉ nằm trong bộ nhớ (Android Developers, n.d.-g).



Luồng thuê xe giữa hai Activity

Cách duy nhất để dữ liệu quay về Home là một lần Save hợp lệ; mọi đường Back đều trả về lệnh huỷ và không thay đổi gì.

Cấu trúc

File	Nhiệm vụ
`model/Car.kt`, `model/Rental.kt`	Lớp dữ liệu, mỗi lớp tự viết `Parcelable`
`model/CarRepository.kt`	5 xe dưới dạng `List` chỉ-đọc, số dư ban đầu 500 và hạn mức 400
`MainViewModel.kt`	Số dư, đơn thuê, yêu thích, từ khoá tìm kiếm, cách sắp xếp, vị trí xe đang hiện
`MainActivity.kt`	Vẽ màn Home từ ViewModel; mở màn Rent và xử lý kết quả
`RentViewModel.kt`	Số ngày, hai Uri ảnh, kiểm tra credit và ảnh
`RentActivity.kt`	Vẽ màn Rent; Photo Picker; Save và Back
`ui/DarkModeToggle.kt`, `ui/FuSnackbar.kt`, `ui/FavouriteAdapter.kt`	Các hàm dùng chung cho cả hai màn

Mọi màn hình đều theo một mẫu: **thao tác của người dùng → hàm trong ViewModel → `render()`**. `render()` vẽ lại toàn bộ từ ViewModel nên giao diện không thể lệch khỏi dữ liệu, và sau khi Activity bị tạo lại (dark mode, xoay màn hình) chỉ cần một lần gọi là màn hình hiện đúng như cũ.

Dữ liệu gốc không bao giờ bị sửa. `displayedCars()` mỗi lần tạo một danh sách mới: xe chưa thuê → khớp từ khoá → sắp xếp bằng `sortedBy` / `sortedByDescending`. Next quay vòng bằng `(index + 1) % size`.

Các thành phần của Intent

```
fun newIntent(context: Context, car: Car, balance: Int): Intent =
    Intent(context, RentActivity::class.java)   // tường minh: ghi rõ class đích
        .putExtra(EXTRA_CAR, car)                // extra dạng Parcelable
        .putExtra(EXTRA_BALANCE, balance)        // extra kiểu nguyên thủy
```

Thành phần	Trong Fake Uber	Mục đích
Context	`MainActivity`	Nơi gửi Intent đi
Component (class đích)	`RentActivity::class.java`	Biến nó thành Intent **tường minh**: Android mở thẳng class đó, không phải tìm app phù hợp, nên không cần action, category hay data URI (Android Developers, n.d.-c)
Extras	`EXTRA_CAR` (`Car` Parcelable), `EXTRA_BALANCE` (`Int`)	Dữ liệu màn thứ hai cần, dạng cặp khoá–giá trị trong Bundle
Launcher	`registerForActivityResult(StartActivityForResult())`	Mở màn Rent và nhận kết quả (Android Developers, n.d.-b) (thay cho `startActivityForResult` đã bị khai tử)
Result code	`RESULT_OK` / `RESULT_CANCELED`	Phân biệt Save và Back
Result extras	`EXTRA_RENTAL` (`Rental` Parcelable) hoặc `EXTRA_CAR`	Đơn thuê, hoặc tên xe cho thông báo "not saved"

Ở phía nhận, `IntentCompat.getParcelableExtra(intent, EXTRA_CAR, Car::class.java)` đọc xe một cách an toàn về kiểu, chạy được trên cả Android cũ lẫn mới. Back có ba lối vào (nút Back, mũi tên và cử chỉ back của hệ thống qua `OnBackPressedDispatcher`), cả ba đều gọi chung `cancelRental()`, nên không có đường nào lỡ tay lưu đơn.

Vì sao dùng Parcelable

`Parcelable` là định dạng tuần tự hoá riêng của Android để truyền object giữa các thành phần: object ghi các field vào một `Parcel` phẳng, và `CREATOR` dựng lại object ở phía bên kia (Android Developers, n.d.-d).

```
override fun writeToParcel(parcel: Parcel, flags: Int) {
    parcel.writeInt(id)
    parcel.writeString(brand)
    parcel.writeString(model)
    parcel.writeInt(year)
    // … các field còn lại, luôn theo cùng một thứ tự
}
```

Ưu điểm so với `Serializable` của Java:

- **Nhanh và nhẹ hơn.** Không dùng reflection và ít tạo object tạm, điều quan trọng trên main thread và khi Intent đi qua Binder giữa các tiến trình.
- **Tường minh.** Mình quyết định chính xác cái gì được ghi; không có field riêng tư nào bị ghi nhầm.
- **Lồng nhau gọn gàng.** `Rental.writeToParcel()` gọi `car.writeToParcel()` trước, nên một Parcelable chứa được một Parcelable khác.
- **Đọc an toàn về kiểu** với `IntentCompat.getParcelableExtra(…, Car::class.java)`.

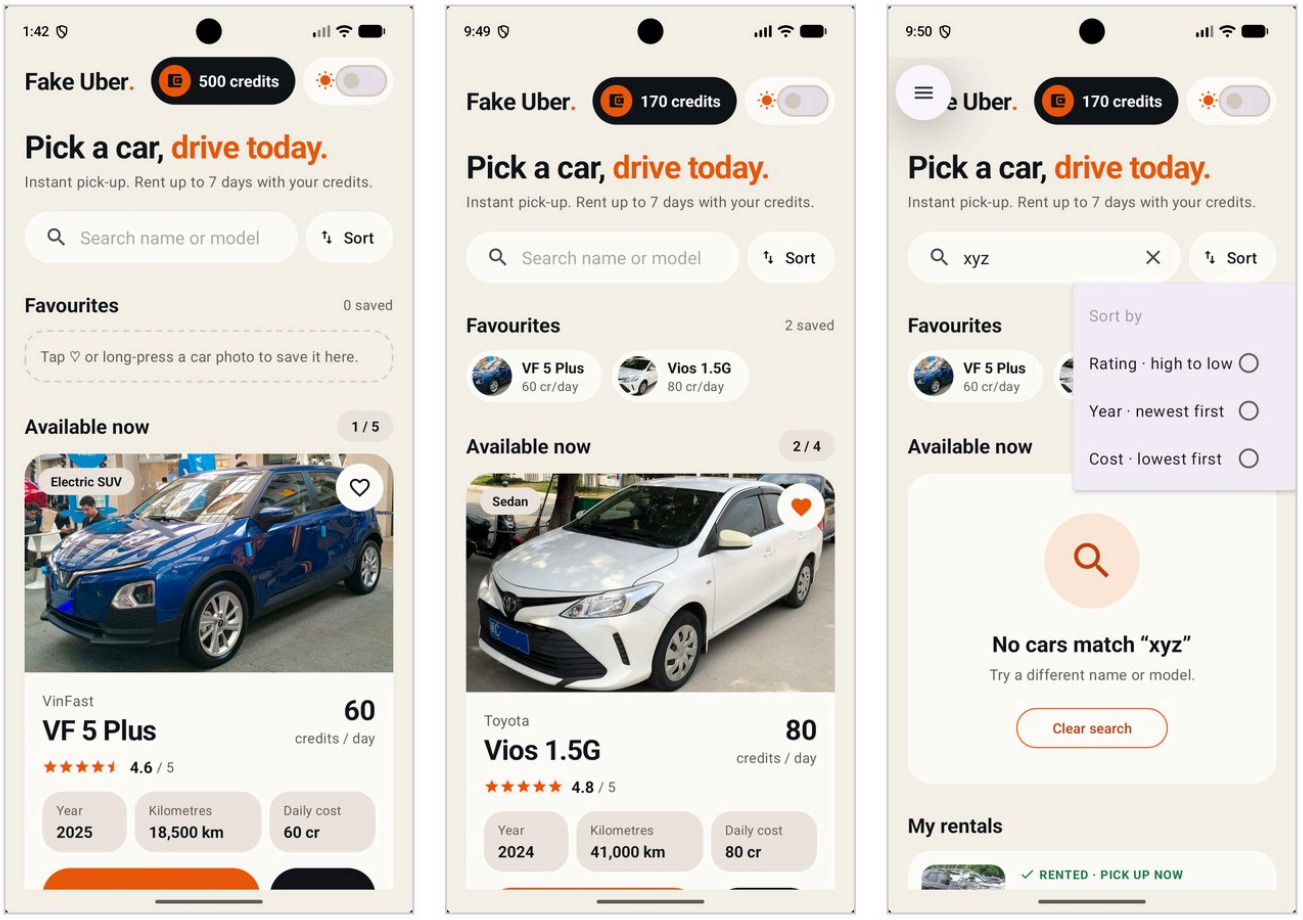
Cái giá là code dài hơn và một quy tắc nghiêm ngặt: phải đọc đúng thứ tự đã ghi, nếu không dữ liệu sẽ lặng lẽ rơi vào sai field. Mình tự viết cả hai lớp thay vì dùng `@Parcelize`, một phần để hiểu cơ chế này, một phần vì version của plugin `kotlin-parcelize` phải khớp với Kotlin đi kèm Android Gradle Plugin 9.

6. Demo các tính năng

App hoàn chỉnh khớp với thiết kế Layout A trên mọi màn hình; các hình dưới đây là ảnh chụp trên emulator Pixel 7.

Xem xe: từng xe một, yêu thích, sắp xếp

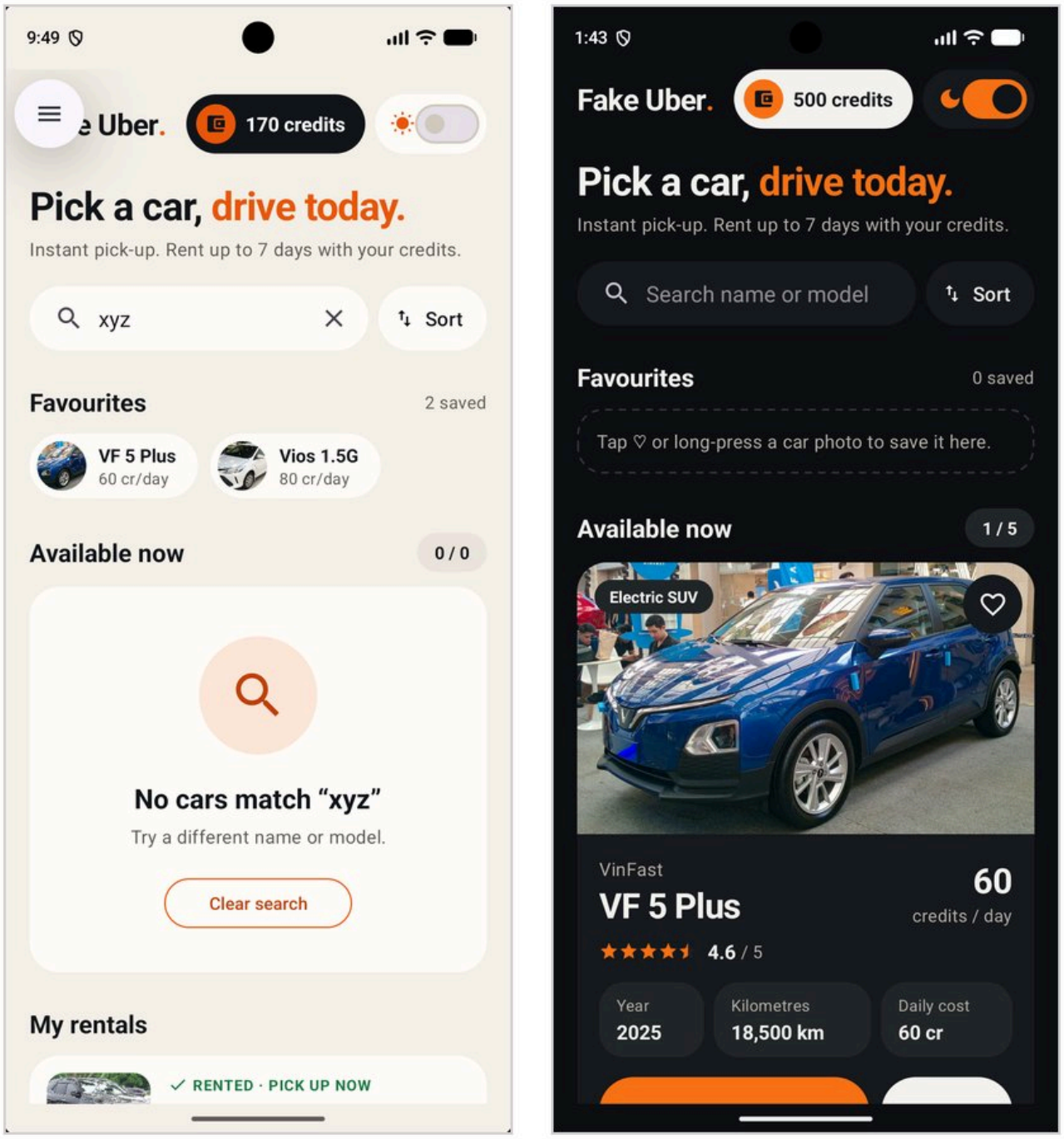
Màn Home hiện từng xe một với hãng, model, RatingBar, năm, số km và giá mỗi ngày dưới ảnh, kèm bộ đếm "1 / 5" và nút Next quay vòng. Trái tim (hoặc nhấn giữ ảnh) thêm xe vào dải Favourites; chạm vào chip sẽ quay lại xe đó. Nút Sort mở PopupMenu với ba kiểu sắp xếp đề yêu cầu.



Hình 3. Màn Home, Favourites đã lưu hai xe, và menu Sort.

Tìm kiếm

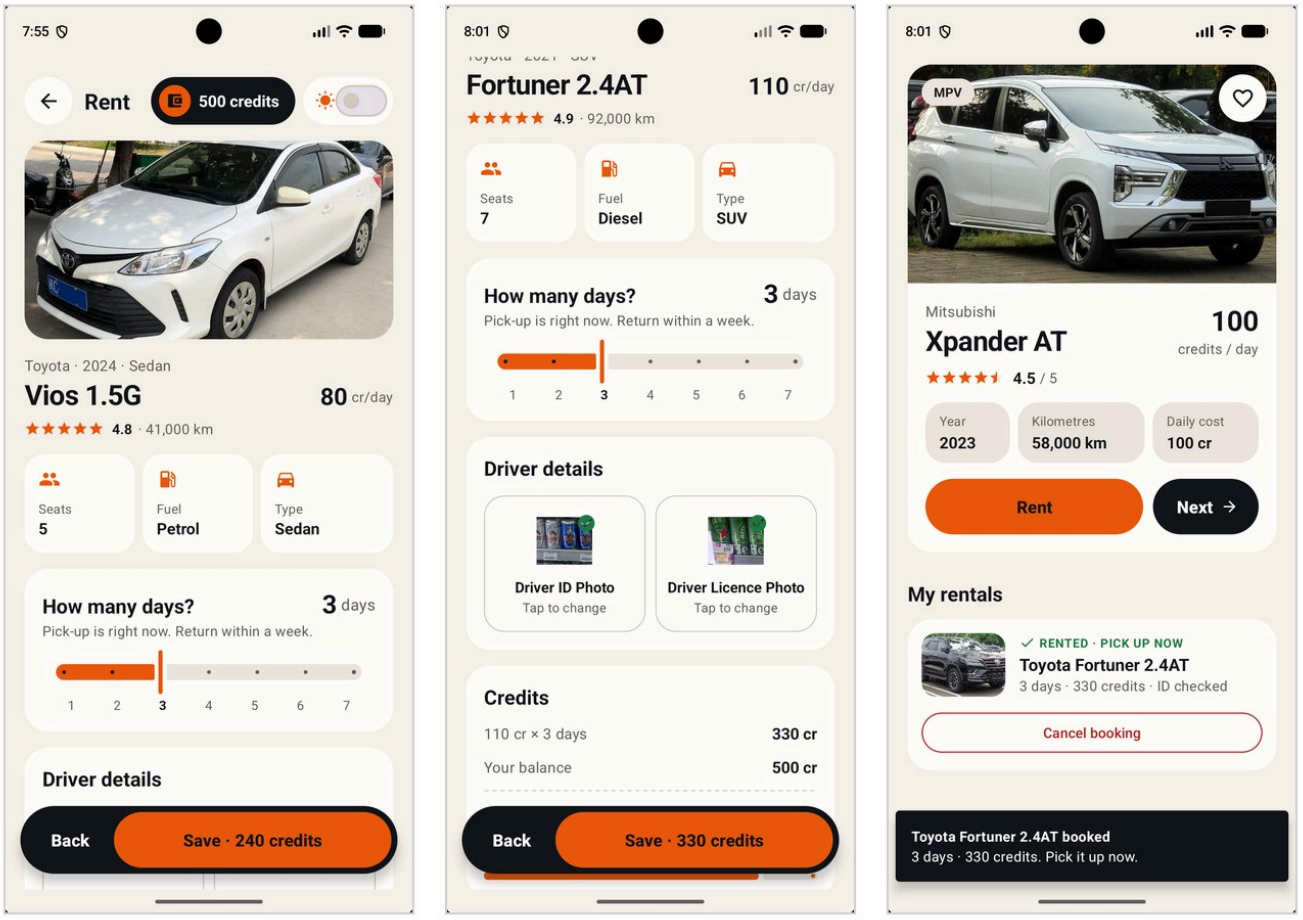
SearchView lọc theo hãng hoặc model sau mỗi ký tự gõ. Khi không có xe nào khớp, thẻ xe được thay bằng "No cars match “xyz”" kèm nút Clear search, bộ đếm hiện "0 / 0".



Hình 4. Tìm kiếm không có kết quả, và màn Home ở chế độ tối.

Thuê xe

Màn Rent hiện lại xe kèm ba thẻ Seats, Fuel và Type, Slider chọn 1–7 ngày, hai ô giấy tờ mở Photo Picker và thẻ Credits. Sau khi Save, màn Home hiện Snackbar "… booked", số dư giảm, xe rời Available now và xuất hiện trong My rentals với nút Cancel booking.

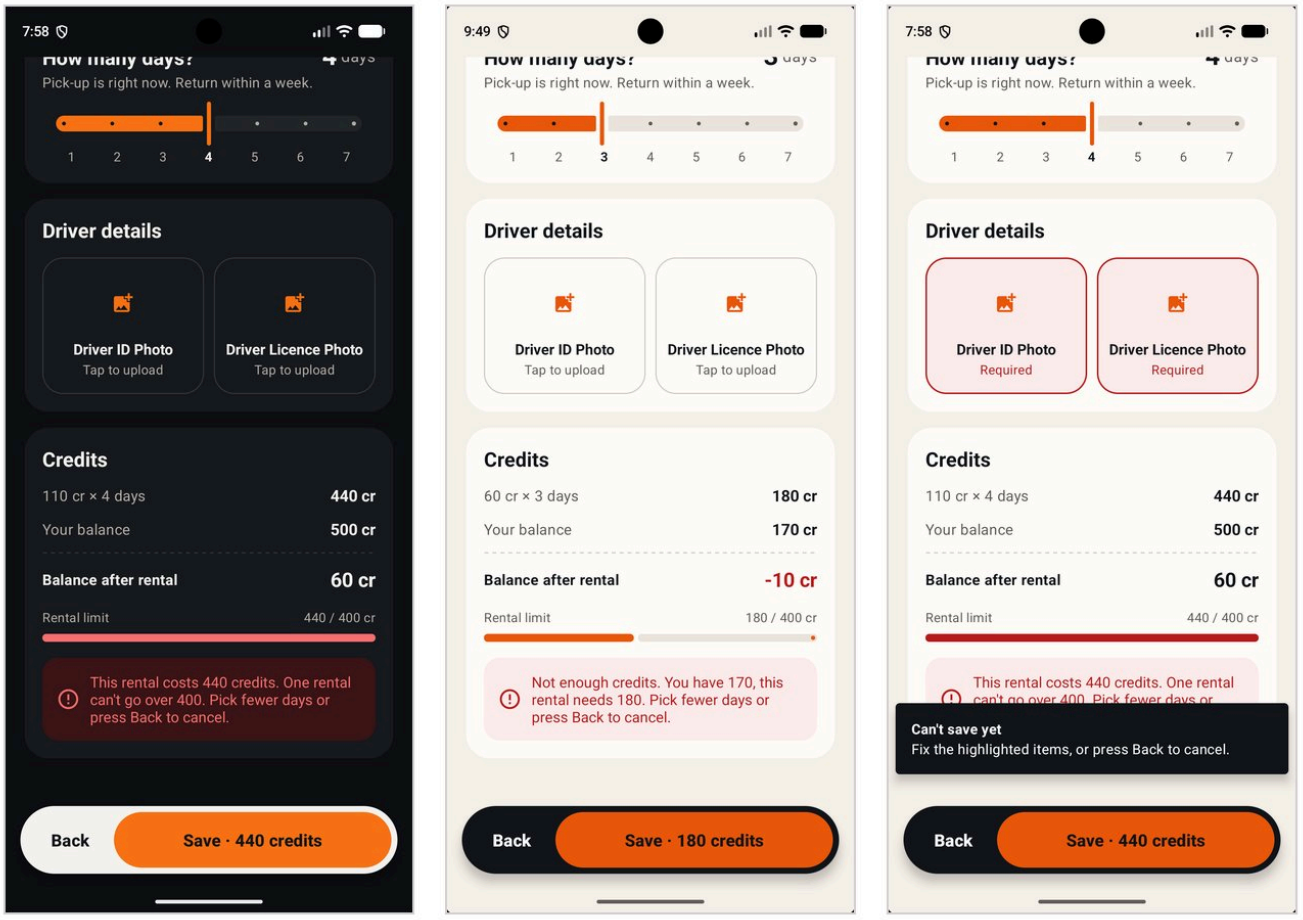


Hình 5. Màn Rent, đã thêm đủ hai ảnh giấy tờ, và đơn thuê được xác nhận trên Home.

Kiểm tra lỗi

Ba trường hợp chặn Save, và trong mọi trường hợp người dùng vẫn ở lại màn Rent cho tới khi sửa xong hoặc bấm Back:

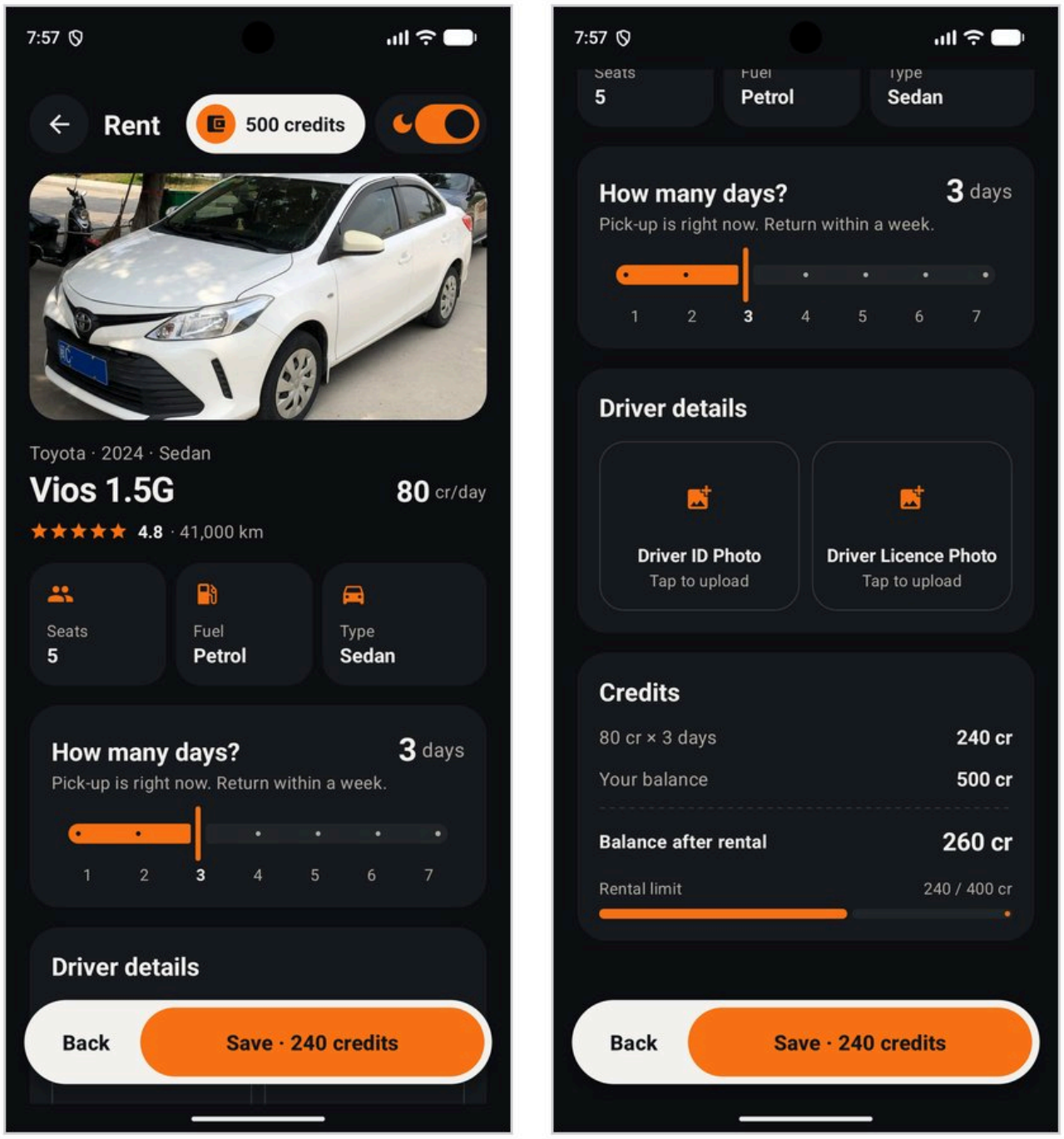
Lỗi	Ví dụ	Người dùng thấy gì
Vượt hạn mức 400	Fortuner × 4 ngày = 440	Thanh hạn mức đỏ và "This rental costs 440 credits. One rental can't go over 400…"
Không đủ credit	Sau đơn 330 credit, VF 5 × 3 ngày = 180 > 170	Số dư sau khi thuê −10 cr màu đỏ và "Not enough credits. You have 170…"
Thiếu ảnh giấy tờ	Bấm Save khi chưa có ảnh	Hai ô ảnh viền đỏ chữ "Required" và Snackbar "Can't save yet"



Hình 6. Vượt hạn mức, không đủ credit, và thiếu ảnh giấy tờ.

Dark mode trên cả hai màn

Switch ở header của Home và Rent đổi chế độ cho cả app. Mỗi màu có một giá trị sáng và một giá trị tối cùng tên, nên cả hai màn đổi giao diện cùng lúc, còn xe đang xem, số ngày và ảnh đã chọn vẫn được giữ.



Hình 7. Màn Rent ở chế độ tối.

Đối chiếu yêu cầu của đề

Yêu cầu	Đáp ứng ở đâu
Năm xe, dữ liệu trong bộ nhớ, thông tin dưới ảnh	`CarRepository`; thẻ xe ở Home (Hình 3)
Từng xe một, có nút Next	Next quay vòng; bộ đếm "1 / 5"

Yêu cầu	Đáp ứng ở đâu
Chỉ dùng ảnh miễn phí	Wikimedia Commons, CC BY-SA 4.0 (mục 11)
Rent mở màn mới với Parcelable	Intent tường minh mang `Car` (mục 5)
Xe đã thuê rời danh sách; huỷ thì quay lại	My rentals + Cancel booking (Hình 5)
500 credit, ⩽ 400 mỗi đơn, số dư ở header	Ô credit trên cả hai màn; kiểm tra trực tiếp (Hình 6)
Save lưu, Back là huỷ, kết quả hiện ở màn đầu	Activity Result API; Snackbar; My rentals
Toast hoặc Snackbar khi đặt / không đặt	Snackbar hai dòng cho mọi kết quả
Yêu thích bằng tim hoặc nhấn giữ, có mục riêng	Tim + nhấn giữ; dải Favourites
Thanh tìm kiếm và menu Sort (rating, năm, giá) không đổi dữ liệu gốc	SearchView + PopupMenu trên bản sao danh sách
Switch dark mode, style đổi trên ít nhất hai màn	Switch trên Home và Rent (Hình 4 và 7)
Ít nhất hai widget không phải TextView	RatingBar, Slider (thêm Switch, LinearProgressIndicator)
Kiểm tra lỗi, chặn quay về cho tới khi sửa hoặc huỷ	`trySave()` trả về null và màn hình vẫn mở
Ít nhất hai style dùng ở ít nhất hai chỗ	Sáu style, mỗi style dùng trên cả hai màn (mục 7)

7. Style, theme, dark mode và accessibility

App định nghĩa sáu style dùng lại (cộng một style con), và style nào cũng được dùng trên cả hai màn, vượt mức tối thiểu "hai style ở hai chỗ" của đề.

Style	Định nghĩa	Dùng ở Home	Dùng ở Rent
`TextAppearance.FakeUber.Title`	28sp đậm, màu chữ chính	Logo, tiêu đề lớn, tên xe	Tên xe

Style	Định nghĩa	Dùng ở Home	Dùng ở Rent
`TextAppearance.FakeUber.Section`	20sp đậm	Favourites, Available now, My rentals	Tiêu đề Rent, How many days?, Driver details, Credits
`TextAppearance.FakeUber.Secondary`	14sp, xám phụ	Tên hãng, "credits / day", nhãn, gợi ý	Dòng thông tin, chữ hướng dẫn, các dòng credit
`Widget.FakeUber.CreditPill`	Nền đảo màu bo tròn, icon ví, số dư đậm	Header	Header
`Widget.FakeUber.Button.Primary`	Nền cam, chữ tối, cao 56dp, bo tròn	Rent	Save
`Widget.FakeUber.Button.Inverse`	Kế thừa Primary, chỉ đổi màu	Next	Back
`Widget.FakeUber.Card`	Màu nền thẻ, góc 24dp, không đổ bóng	Thẻ xe, thẻ trạng thái trống, thẻ đơn thuê	Thẻ ảnh, số ngày, giấy tờ, credit

Một theme, hai file màu. `Theme.FakeUber` kế thừa `Theme.Material3.DayNight.NoActionBar` và gán bảng màu vào các vai trò màu của Material 3 (`colorPrimary` , `colorSurface` , `colorSecondaryContainer` , `colorSurfaceInverse` ...). Mỗi màu có hai bản cùng tên trong `values/colors.xml` và `values-night/colors.xml` , nên layout chỉ cần ghi `@color/fu_text` hay `@color/fu_bg` và Android tự chọn đúng giá trị. Nhờ gán vai trò màu, Slider, Switch, thanh tiến trình và Snackbar tự đổi màu mà không cần code riêng.

Công tắc. `DarkModeToggle.bind()` dùng chung cho cả hai màn. Hàm đọc chế độ hiện tại từ configuration, gán trạng thái cho Switch trước rồi mới gắn listener gọi `AppCompatDelegate.setDefaultNightMode()` . Lệnh này tạo lại mọi Activity đang mở, nên toàn bộ trạng thái năm trong ViewModel. Switch được đặt `saveEnabled="false"` và listener bỏ qua giá trị trùng chế độ hiện tại: thiếu hai lớp bảo vệ này, màn Home và Rent khôi phục trạng thái Switch cũ của nhau và đẩy app qua lại giữa sáng và tối mãi (mục 9, khó khăn số 2).

Danh sách accessibility

- Độ tương phản chữ tối thiểu 4,5:1 ở cả hai giao diện (mục 4)
- Mọi thành phần bấm được tối thiểu 48 × 48dp
- Content description cho ảnh xe, trái tim, ô giấy tờ và Cancel booking
- Tiêu đề được đánh dấu `accessibilityHeading` để điều hướng bằng TalkBack
- Hộp lỗi là live region
- Nhãn TalkBack riêng cho thao tác nhấn giữ
- Phản hồi ở đáy màn hình bằng Snackbar

8. Kiểm thử

Cả 10 test giao diện Espresso đều đạt trong 43 giây trên emulator Pixel 7, và một lượt test tay toàn bộ tính năng ở cả hai giao diện không phát hiện lỗi nào sau các bản sửa ở mục 9.

Các test theo đúng lời khuyên của đề: test chữ và nút bấm thay vì RatingBar, và bỏ code lặp. Mọi lệnh Espresso nằm trong một class **robot** là `FakeUberRobot`, nên mỗi test đọc như các bước người dùng thao tác:

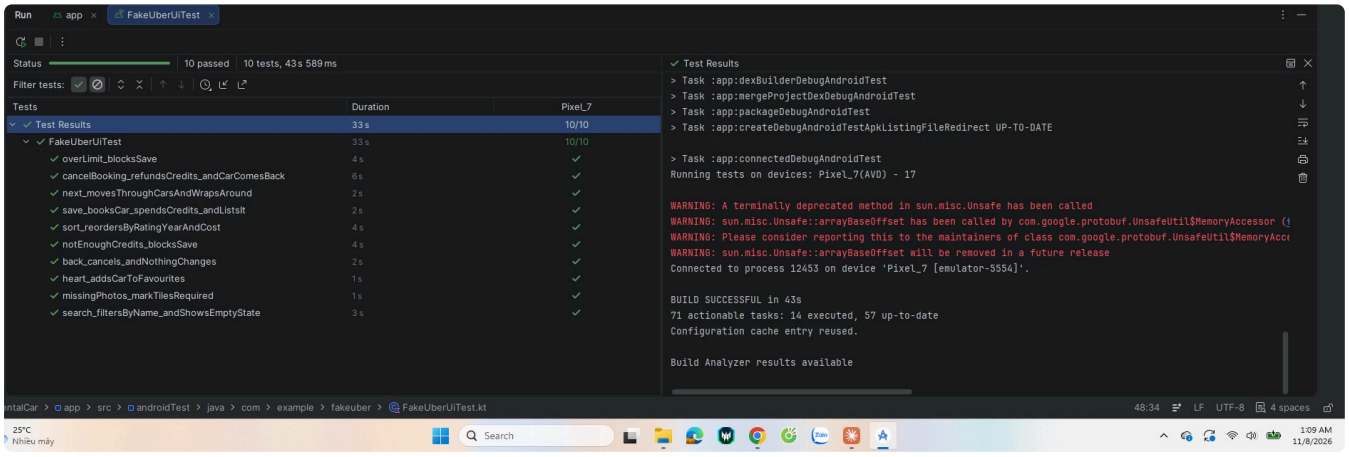
```
@Test
fun notEnoughCredits_blocksSave() {
    with(robot) {
        next(4)
        bookCurrentCar(days = 3)   // Fortuner 330 → còn 170
        checkCredits(170)
        next()                     // quay vòng về VF 5
        rent()
        setDays(3)                 // 60 × 3 = 180 > 170
        checkTextContains(R.id.tvError, "Not enough credits")
    }
}
```

Test	Chứng minh điều gì
`next_movesThroughCarsAndWrapsAround`	Next đi 1/5 → 2/5 … rồi quay về 1/5
`save_booksCar_spendsCredits_andListsIt`	VF 5 × 3 ngày: số dư 320, "1 / 4", có nút Cancel booking
`back_cancels_andNothingChanges`	"Rental not saved", số dư vẫn 500
`overLimit_blocksSave`	Fortuner × 4 = 440 báo lỗi hạn mức và vẫn ở màn Rent

Test	Chứng minh điều gì
`notEnoughCredits_blocksSave`	180 > 170 báo "Not enough credits"
`missingPhotos_markTilesRequired`	Cả hai ô ảnh báo "Required"
`cancelBooking_refundsCredits_andCarComesBack`	Hoàn 120 credit, về lại 500 và "1 / 5"
`search_filtersByName_andShowsEmptyState`	"toyota" → 1 / 2, "xyz" → 0 / 0, Clear search → 1 / 5
`sort_reordersByRatingYearAndCost`	Fortuner đầu tiên theo rating, VF 5 theo năm và theo giá
`heart_addsCarToFavourites`	"0 saved" → "1 saved", hiện dải Favourites

Cần thêm ba phần hỗ trợ cho test:

- **Giả lập Photo Picker.** `espresso-intents` trả lời ngay Intent của Photo Picker bằng một ảnh có sẵn trong app, nên test không mở thư viện ảnh thật (Android Developers, n.d.-a).
- **Hai ViewAction tự viết.** `scrollTo()` của Espresso không hỗ trợ `NestedScrollView` và cũng không có thao tác cho Material Slider, nên robot có thêm `nestedScrollTo()` và `setSliderValue()`.
- **Chờ có điều kiện cho Snackbar.** Snackbar xếp hàng, nên `checkSnackbar()` kiểm tra lại mỗi 100ms trong tối đa 5 giây thay vì chờ một khoảng cố định.



Hình 8. Kết quả test trong Android Studio: 10 test đạt trong 43 giây.

9. Khó khăn, khám phá và bài học

Những vấn đề khó nhất không năm ở tính năng mà ở vòng đời của Android: việc tạo lại Activity khi đổi dark mode gây ra ba lỗi khác nhau, và test giao diện đã phát hiện một lỗi UX thật mà test tay bỏ sót. Mình ghi nhật ký phát triển suốt quá trình; dưới đây là những khó khăn dạy mình nhiều nhất.

Khó khăn

#	Vấn đề	Nguyên nhân	Cách sửa
1	Số dư, đơn thuê và xe đang xem bị reset khi đổi dark mode	`setDefaultNightMode()` huỷ và tạo lại Activity	Chuyển toàn bộ trạng thái vào `MainViewModel` / `RentViewModel`; một hàm `render()` vẽ lại màn hình
2	Đổi sang chế độ sáng ở màn Rent làm app nhấp nháy giữa hai giao diện mãi	Khi tạo lại, Android khôi phục trạng thái từng view sau `onCreate()`; trạng thái Switch cũ ở Home và Rent kích hoạt listener và đảo ngược nhau	`saveEnabled="false"` cho Switch, và listener bỏ qua giá trị trùng chế độ hiện tại
3	Sau khi đổi dark mode, thẻ nhảy về xe đầu tiên khi đang tìm kiếm	SearchView khôi phục chữ và gọi lại `onQueryTextChange`, làm reset vị trí	`setQuery()` bỏ qua nếu chữ không đổi
4	Slider và màn hình có thể cập nhật lẫn nhau thành vòng lặp	`render()` gán giá trị Slider và listener của Slider lại gọi `render()`	Listener chỉ xử lý khi giá trị khác ViewModel
5	Mỗi ảnh xe tốn khoảng 19 MB RAM	Ảnh JPG 1024px trong `drawable/` bị coi là mdpi và phóng to 2,6 lần	Chuyển sang `drawable-nodpi/`; ảnh người dùng chọn chỉ đọc bản khoảng 200px
6	Thanh Slider hiện màu tím mặc định của Material	Theme chưa định nghĩa `colorSecondaryContainer`	Gán mọi vai trò màu Material 3 vào bảng màu
7	Nội dung trượt dưới thanh trạng thái khi cuộn	Inset edge-to-edge được áp dụng làm padding với `clipToPadding="false"`	`clipToPadding="true"` cho màn Home

#	Vấn đề	Nguyên nhân	Cách sửa
8	10 test chạy mất 8 phút	Bộ giả lập Intent bắt mọi Intent ra ngoài app, kể cả màn phụ của ActivityScenario, nên mỗi test phải chờ timeout 45 giây	Chỉ giả lập các action của Photo Picker
9	Test huỷ đơn bị lỗi	Snackbar "booked" che nút Cancel booking ở cuối trang, cú chạm của Espresso rơi vào Snackbar; người dùng thật cũng gặp vấn đề này	Thêm 120dp khoảng trống ở cuối Home để nút cuối cuộn lên được phía trên Snackbar; test chờ Snackbar biến mất
10	Viết `Parcelable` đúng	Đọc field khác thứ tự lúc ghi sẽ làm hỏng dữ liệu mà không báo lỗi	Viết `writeToParcel` và constructor đọc cạnh nhau, đối chiếu từng field

Khám phá ngoài nội dung các tuần học

- Activity Result API (`registerForActivityResult`) thay cho `startActivityForResult` đã bị khai tử
- Photo Picker của hệ thống, không cần quyền truy cập bộ nhớ
- ViewModel cho trạng thái tồn tại qua thay đổi cấu hình
- Vai trò màu Material 3 với `values-night` để làm dark mode bằng một theme
- ViewBinding với `<include>` cho ô chỉ số, thẻ thông số và ô giấy tờ dùng lại
- RecyclerView với Adapter và ViewHolder cho dải Favourites
- `espresso-intents` , ViewAction tự viết và mẫu robot

Bài học

- Việc tạo lại Activity là chuyện bình thường, không phải trường hợp hiếm: đừng để trạng thái trong Activity, và chặn mọi listener có tác dụng phụ.
- Accessibility rẻ nhất khi được đưa vào bảng màu và kích thước ngay từ đầu.
- Test giao diện tìm ra cả lỗi UX thật, không chỉ lỗi code (khó khăn số 9).
- Phác thảo widget trước khi code khiến việc xây dựng app chủ yếu chỉ là làm theo kế hoạch.

10. Nhật ký thời gian

Dự án kéo dài từ 4 đến 8/11/2026, khoảng nửa đầu dành cho lập kế hoạch, nghiên cứu và phác thảo, nửa sau dành cho xây dựng và kiểm thử.

Ngày	Công việc
4/11/2026	Đọc đề, lập kế hoạch (Google Sheet), nghiên cứu UI/UX ban đầu
5–6/11	Các bản phác thảo v4 đến v8 (14 màn), bản Figma, ghi chú accessibility
7/11	Báo cáo UI/UX cuối (27 nguồn), thiết kế UX (user story, use case), nghiên cứu thị trường cho 5 xe và ảnh miễn phí, Layout A và B dạng low-fi
7/11, buổi tối	Tạo project Android Studio; model dữ liệu, lớp Parcelable, repository trong bộ nhớ, ảnh; màu, theme và style; màn Home và công tắc dark mode
8/11, rạng sáng	Màn Rent với Intent, Parcelable, Activity Result API, kiểm tra lỗi và Photo Picker; yêu thích, tìm kiếm, sắp xếp, My rentals và huỷ đơn; test tay toàn bộ tính năng
8/11, rạng sáng	10 test Espresso: lần đầu 9/10 trong 8 phút 5 giây, sau khi sửa bộ giả lập và lỗi Snackbar thì 10/10 trong 43 giây
8/11, buổi sáng	Sửa lỗi nhấp nháy dark mode ở màn Rent; chụp ảnh màn hình; viết báo cáo này

11. Tài liệu tham khảo, nguồn ảnh và việc dùng AI

Tài liệu tham khảo

Tài liệu tham khảo trình bày theo chuẩn APA 7. Trang không có ngày xuất bản ghi là n.d. kèm ngày truy cập.

Android Developers. (n.d.-a). *Espresso-Intents*. Truy cập ngày 8/11/2026, từ https://developer.android.com/training/testing/espresso/intents

Android Developers. (n.d.-b). *Get a result from an activity*. Truy cập ngày 8/11/2026, từ https://developer.android.com/training/basics/intents/result

Android Developers. (n.d.-c). *Intents and intent filters*. Truy cập ngày 8/11/2026, từ https://developer.android.com/guide/components/intents-filters

Android Developers. (n.d.-d). *Parcelable*. Truy cập ngày 8/11/2026, từ https://developer.android.com/reference/android/os/Parcelable

Android Developers. (n.d.-e). *Photo picker*. Truy cập ngày 8/11/2026, từ https://developer.android.com/training/data-storage/shared/photopicker

Android Developers. (n.d.-f). *Toasts overview*. Truy cập ngày 8/11/2026, từ https://developer.android.com/guide/topics/ui/notifiers/toasts

Android Developers. (n.d.-g). *ViewModel overview*. Truy cập ngày 8/11/2026, từ https://developer.android.com/topic/libraries/architecture/viewmodel

Budiu, R. (2018). *Carousels on mobile devices*. Nielsen Norman Group. https://www.nngroup.com/articles/mobile-carousels/

Chen, D. (n.d.). *MAZDA3 SEDAN (BP) China* [Ảnh]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:MAZDA3_SEDAN_(BP)_China.jpg

Febrian, A. (n.d.). *2021 Toyota Fortuner 2.4 TRD Sportivo (Indonesia) front view* [Ảnh]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:2021_Toyota_Fortuner_2.4_TRD_Sportivo_(Indonesia)_front_view.jpg

JamesYoung8167. (2022). *Toyota Vios XP150 Facelift Shishi 01 2022-09-20* [Ảnh]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:Toyota_Vios_XP150_Facelift_Shishi_01_2022-09-20.jpg

Llamas, E. (n.d.). *2025 VinFast VF 5 in Vinfast Blue, front left* [Ảnh]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:2025_VinFast_VF_5_in_Vinfast_Blue,_front_left.jpg

Material Components for Android. (n.d.). *Snackbar* [Tài liệu hướng dẫn]. GitHub. Truy cập ngày 8/11/2026, từ https://github.com/material-components/material-components-android/blob/master/docs/components/Snackbar.md

Neusesser, T., & Sunwall, E. (2023). *Error-message guidelines*. Nielsen Norman Group. https://www.nngroup.com/articles/error-message-guidelines/

Parhi, P., Karlson, A. K., & Bederson, B. B. (2006). Target size study for one-handed thumb use on small touchscreen devices. In *Proceedings of the 8th Conference on Human-Computer Interaction with Mobile Devices and Services* (pp. 203–210). Association for Computing Machinery. https://doi.org/10.1145/1152215.1152260

W3C. (2023). *Understanding success criterion 1.4.3: Contrast (minimum)*. https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html

オーバードライブ83. (2022). *2022 Mitsubishi Xpander Ultimate 1.5 NC2W (20220221)* [Ảnh]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:2022_Mitsubishi_Xpander_Ultimate_1.5_NC2W_(20220221).jpg

Nguồn ảnh

Tất cả ảnh xe lấy từ Wikimedia Commons theo giấy phép CC BY-SA 4.0, chỉ thu nhỏ kích thước.

Xe	File trên Commons	Tác giả
VinFast VF 5 Plus	2025 VinFast VF 5 in Vinfast Blue, front left	Ethan Llamas
Toyota Vios 1.5G	Toyota Vios XP150 Facelift Shishi 01 2022-09-20	JamesYoung8167
Mazda3 1.5	MAZDA3 SEDAN (BP) China	Dinkun Chen
Mitsubishi Xpander AT	2022 Mitsubishi Xpander Ultimate 1.5 NC2W (20220221)	オーバードライブ83
Toyota Fortuner 2.4AT	2021 Toyota Fortuner 2.4 TRD Sportivo (Indonesia) front view	Andra Febrian

Icon là Material Icons (giấy phép Apache 2.0).

Việc sử dụng công cụ AI

Mình dùng Claude (Anthropic) làm trợ lý cho nghiên cứu UI/UX, phác thảo low-fi, soạn code và soạn báo cáo này. Mình tự đọc lại, build và chạy toàn bộ code trong Android Studio, test tay từng tính năng trên emulator, chạy bộ test Espresso, đồng thời phát hiện và báo các lỗi được mô tả ở mục 9.